PDFStudio

What the specification builds, explained

1. What PDFStudio is

The document is a detailed specification (a prompt) for building **PDFStudio**, a commercial-grade platform for generating and managing PDF documents. It is written as instructions for an AI or a development team: act as a group of architects and design the whole system in phases.

Think of it as your own self-hosted version of services like DocRaptor or PDFShift, reusable across all of your company's products. A business application (for example an invoicing system) sends data plus a template ID, and PDFStudio returns a finished PDF.

```
Template (HTML with {{placeholders}})  +  JSON data   -->   PDF file
```

Example: an invoice template containing `{{customer.name}}` and `{{invoice.total}}` is combined with `{"customerName": "ABC Technologies", "amount": 12500}`. The result is a stored, downloadable PDF.

2. What it should do

Generate PDFs

- Synchronously for small documents (result returned immediately).
- Asynchronously for large or bulk jobs: you get a job ID, then poll for status or receive a webhook.

Manipulate PDFs

- Merge, split, rotate, reorder, extract pages, stamp, watermark, edit metadata.
- Password protection, encryption and permission restrictions.
- Future extension points: digital signatures, PDF/A, forms, OCR, document comparison.

Manage templates

- Create, edit, version, publish, clone, preview, activate and deactivate.
- Variables, loops, conditionals, tables, images, headers/footers, page numbers, localization and formatting.
- Protection against template injection and unsafe content.

Manage documents

- Generated PDFs are stored, versioned, and can be downloaded, shared, archived or expired.

3. Who uses it

Client	Purpose
Angular 21 web app	Admin and user UI: dashboard, template designer, document lists, job monitor, API key management, user/role administration.
WPF Windows app	Desktop tool with online mode (via API) and offline local PDF processing, batch processing and printer management.
.NET MAUI mobile app	Android/iOS: generate, preview, download and share PDFs.
.NET SDK	A client library so other developers can call PDFStudio in a few lines of C#.

Client	Purpose
External applications	Call the Public API with API keys or OAuth.

4. Two APIs

- **Public API** (`/api/v1/public/*`): for external customers and apps. Authenticated with API keys, OAuth/OIDC or JWT; scoped (for example pdf:generate, template:read); rate-limited.
- **Private API** (`/api/v1/private/*`): for your own web and Windows apps and internal services. Covers administration of tenants, users, storage, audit and system settings. It must **never** be exposed to the public internet.

5. Enterprise requirements

- **Multi-tenancy:** many organizations on one system. Every business table carries OrganizationId, and one organization must never see another's templates, documents, files, keys, jobs or audit records.
- **Security:** hashed API keys (the secret is shown only once), RBAC, rate limiting, SSRF, path-traversal and template-injection protection, account lockout, audit and security logs, secure headers.
- **Database:** SQL Server and PostgreSQL schemas, accessed through plain ADO.NET with repositories (no Entity Framework).
- **Swappable parts:** IPdfEngine, IStorageProvider (local disk, Azure Blob, S3) and the data layer are all interfaces, so no implementation is locked in.
- **Webhooks:** events such as pdf.generated or job.failed, delivered securely with retries.
- **Licensing module:** Trial, Community, Professional and Enterprise tiers with limits on PDF count, API calls, storage, users and so on.
- **Operations:** Docker and docker-compose, Caddy reverse proxy, CI/CD pipeline, unit/integration/security/UI tests, horizontal scaling with several API servers and PDF workers.

6. Architecture in plain terms

A **modular monolith** with Clean Architecture (not microservices at first), designed so it can be split later.

```
Clients (Web / Windows / Mobile / External apps)
            |
        API Gateway
            |
   Public API   |   Private API
            |
   Application layer (business logic)
            |
   Template Engine | PDF Engine | Document Engine
            |
   Job / Worker engine (background processing)
            |
   SQL Server / PostgreSQL        File storage
```

The roughly 16 projects under src/ each have one responsibility: Domain holds core entities, Application holds use cases, Infrastructure handles database and external access, PdfEngine generates PDFs, Security handles authentication and hashing, and so on.

7. How it is meant to be built

The specification says explicitly **not** to generate everything at once. It defines 18 phases:

Phases	Focus
1-3	Architecture and solution structure, database, domain and application layers
4-6	Authentication/security, Public API, Private API
7-10	PDF engine abstraction and first implementation, template engine, document management, background jobs
11-13	Angular web, Windows WPF, .NET MAUI mobile
14-18	SDK, Docker, testing, CI/CD, production hardening

The **First Task** at the end asks only for the architecture: diagrams, solution structure, API, database, security, PDF and template design, deployment, phases, risks and technical decisions, followed by a recommended build order with reasons. No implementation code yet.

8. Honest observations

- **It is very large.** Realistically a multi-person, many-month product. The phased approach is the right way to handle it.
- **"Prefer built-in Microsoft libraries" has a catch.** .NET has no built-in PDF generation or manipulation library. You will almost certainly need a third-party engine (for example QuestPDF, PDFsharp, iText, or headless Chromium for HTML to PDF). This is why the spec insists on the IPdfEngine abstraction and on justifying every dependency. It will be the biggest early technical decision.
- **Offline processing in WPF** is only practical if the chosen PDF engine can run locally.
- **Versions:** Angular 21 and .NET 9 are fine targets, but check support lifecycles; .NET 9 is a short-term-support release.

9. Suggested next step

Produce the First Task deliverable: the full architecture document (diagrams, project responsibilities, API, security, database, PDF and template design, risks and phase order). Alternatively, start with one piece such as the PDF engine decision or the database schema.